Informe del proyecto final: Naive Bayes

Clasificación de la velocidad del viento en la ciudad de Medellín

Harlem Josué Hernández Marún

1. INTRODUCCIÓN

En este proyecto quise responder una pregunta sencilla: si conozco la hora, el mes y la estación de medición, ¿puedo saber si el viento en Medellín va a estar bajo, medio o alto? Para eso usé Naive Bayes, que es un modelo de clasificación basado en probabilidades.

El trabajo tiene tres partes. Primero limpié los datos, porque el archivo original venía con registros repetidos y textos escritos de varias formas. Después entrené el modelo y medí qué tan bien clasifica. Al final monté una página web donde cualquier persona puede escoger los valores y ver la predicción.

Escogí Naive Bayes porque las tres variables que tengo son categorías (24 horas, 12 meses y 4 estaciones) y este modelo trabaja contando cuántas veces aparece cada categoría en cada nivel de viento. Eso lo hace fácil de explicar: toda la predicción sale de multiplicar probabilidades.

2. FUENTE DE DATOS

Los datos salen del portal de datos abiertos de Colombia (datos.gov.co), del conjunto "Velocidad del Viento". El conjunto es nacional, así que se filtró por el municipio de Medellín y se guardó en un archivo de Excel con dos hojas, porque Excel no permite más de 1.048.576 filas por hoja.

El archivo tiene 1.986.848 registros y 12 columnas. Cada registro es una medición de velocidad del viento en m/s, tomada cada 10 minutos, con la fecha, el código y el nombre de la estación, el sensor y la ubicación.

Estación	Código	Mediciones	Periodo
Olaya Herrera	0027015330	1.070.453	2014 - 2026
Pajarito	0027015290	572.553	2005 - 2020
Villanueva	0027015300	236.034	2005 - 2014
Santa Elena	0027010810	107.808	2024 - 2026

Tabla 1. Estaciones de Medellín incluidas en el archivo original.

Algo que noté desde el principio es que las estaciones no cubren los mismos años. Villanueva dejó de reportar en 2014 y Santa Elena apenas empezó en 2024, por eso Olaya Herrera tiene más de la mitad de los datos.

3. LIMPIEZA DE LOS DATOS

Antes de limpiar revisé qué problemas tenía el archivo, para no corregir cosas que no estaban dañadas. No encontré valores nulos, pero sí cuatro problemas reales:

- **Registros duplicados.** Había 120.497 filas idénticas en todas las columnas.
- **Textos escritos de varias formas.** El departamento aparecía como "ANTIOQUIA" y "Antioquia", y el municipio como "MEDELLÍN" y "Medellín".
- **Una misma estación con varios nombres.** Olaya Herrera aparecía como "AEROPUERTO OLAYA HERRERA", "APTO OLAYA HERRERA - TX GPRS" y "AEROPUERTO OLAYA HERRERA [27015330]".
- **Valores imposibles.** La velocidad máxima era de 79,2 m/s, que sería un viento más fuerte que el de un huracán.

También noté que Olaya Herrera tiene dos sensores (0103 y 0111) que a veces miden en el mismo momento. Si dejaba las dos mediciones, esa estación quedaba contada dos veces, así que dejé una sola medición por estación y momento.

Paso	Filas que quedan	Eliminadas
Datos originales	1.986.848	-
Quitar nulos	1.986.848	0
Quitar duplicados	1.866.351	120.497
Una medición por estación y momento	1.770.694	95.657
Quitar valores imposibles	1.770.632	62

Tabla 2. Registros que quedan después de cada paso de la limpieza.

Para los textos usé mayúsculas y para las estaciones usé el código, que no cambia, y le asigné un solo nombre a cada una.

En los valores atípicos tuve que tomar una decisión. La regla del rango intercuartílico ponía el límite en 5,1 m/s y eliminaba 64.807 mediciones, pero esas son mediciones reales de viento fuerte, justo las que quiero clasificar como nivel alto. Por eso usé un límite físico: quité solo lo que no puede pasar en Medellín, que son velocidades negativas o de más de 30 m/s (108 km/h).

Después de la limpieza quedaron 1.770.632 registros, el 89 % del archivo original.

4. VARIABLES Y NIVELES DE VIENTO

El conjunto de datos solo trae la fecha, la estación y la velocidad. Por eso las variables para predecir las saqué de lo que había: de la fecha obtuve la hora (0 a 23) y el mes (1 a 12), y la estación la convertí en un número (0 a 3) porque el modelo solo recibe números.

La velocidad es un número continuo y Naive Bayes clasifica en grupos, así que la convertí en tres niveles. Para no inventar los cortes usé la escala de Beaufort, que es la que se usa para describir la fuerza del viento.

Nivel	Velocidad	Registros	%
Bajo	Hasta 1,5 m/s	780.088	44,1
Medio	1,6 a 3,3 m/s	705.484	39,8
Alto	Más de 3,3 m/s	285.060	16,1

Tabla 3. Niveles de viento según la escala de Beaufort.

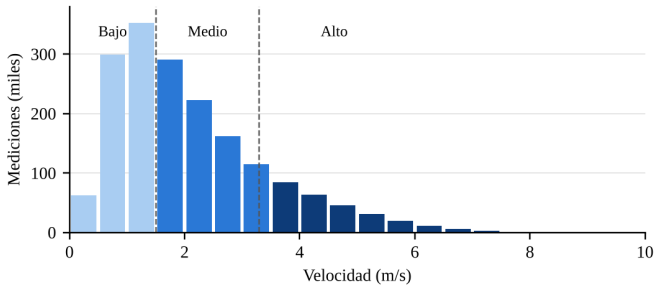


Fig. 1. Distribución de la velocidad del viento y cortes de los tres niveles.

La Fig. 1 muestra que la mayoría de las mediciones está entre 0,5 y 2,5 m/s. El nivel alto es el que menos datos tiene, y eso después se nota en los resultados. También probé agregar el día de la semana, pero la precisión no cambió (51,5 % con y sin esa variable). Tiene sentido: el viento no sabe si es lunes o sábado, así que quedó por fuera.

5. CÓMO SE COMPORTA EL VIENTO

Antes de entrenar revisé si las variables escogidas tenían relación con la velocidad. La hora es la que más influye.

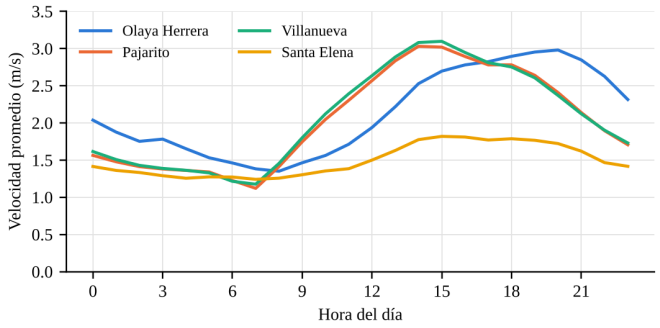


Fig. 2. Velocidad promedio del viento por hora en cada estación.

En las cuatro estaciones el viento es más suave hacia las 7 de la mañana y sube en la tarde. Pajarito y Villanueva tienen su punto más alto cerca de las 3 de la tarde, mientras que Olaya Herrera sigue subiendo hasta las 8 de la noche. Santa Elena es la estación con menos viento a cualquier hora.

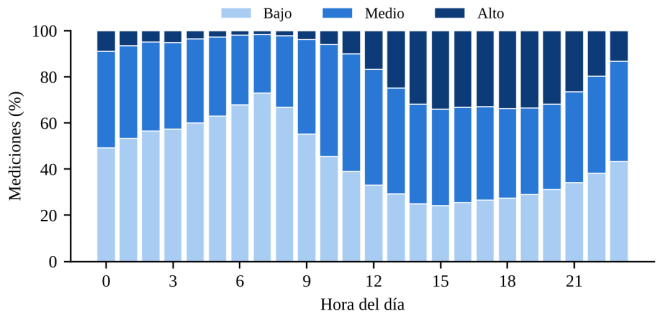


Fig. 3. Porcentaje de mediciones en cada nivel según la hora del día.

La Fig. 3 muestra lo mismo pero con los niveles. A las 7 de la mañana el 73 % de las mediciones son de nivel bajo y menos del 2 % de nivel alto. Entre las 2 de la tarde y las 8 de la noche, una de cada tres mediciones es de nivel alto.

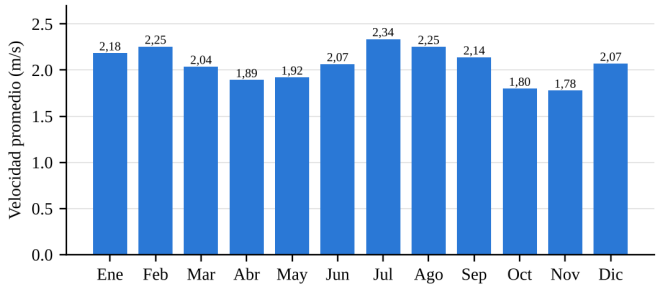


Fig. 4. Velocidad promedio del viento por mes.

Por meses la diferencia es menor. Julio, febrero y agosto son los de más viento y octubre y noviembre los de menos.

6. MODELO NAIVE BAYES

Separé los datos en 70 % para entrenar (1.239.442 registros) y 30 % para probar (531.190 registros). Usé `CategoricalNB` de scikit-learn porque las tres variables son categorías.

Lo que hace el modelo al entrenar es contar. Calcula qué tan común es cada nivel de viento y, dentro de cada nivel, qué tan común es cada hora, cada mes y cada estación. Para predecir multiplica esas probabilidades y escoge el nivel con el resultado más alto.

Un ejemplo con las 3 de la tarde, en agosto, en Olaya Herrera:

Nivel	P(nivel)	P(hora)	P(mes)	P(est.)	Final
Bajo	0,4409	0,0225	0,0669	0,4741	17,7 %
Medio	0,3984	0,0432	0,0818	0,4945	39,1 %
Alto	0,1608	0,0873	0,0999	0,5474	43,2 %

Tabla 4. Cálculo de una predicción con las probabilidades del modelo.

El nivel alto es el menos común en general (16 % de los datos), pero las 3 de la tarde es una hora casi cuatro veces más frecuente dentro del nivel alto que dentro del bajo. Por eso el modelo termina escogiendo Alto. A las 7 de la mañana del mismo mes pasa lo contrario y el modelo responde Bajo con 67,5 %.

7. RESULTADOS

El modelo acierta el 51,5 % de las 531.190 mediciones de prueba. Solo ese número no dice mucho, así que lo comparé con la respuesta más simple posible: decir siempre Bajo, que es el nivel más común. Con eso se acierta el 44,0 %. El modelo mejora esa base en 7,5 puntos, o sea que sí aprendió algo de la hora, el mes y la estación.

Forma de responder	Precisión	Diferencia
Siempre Bajo	44,0 %	-
Modelo Naive Bayes	51,5 %	+7,5 puntos

Tabla 5. Precisión del modelo frente a la respuesta más simple.

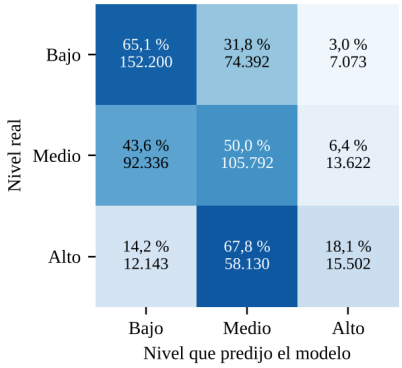


Fig. 5. Aciertos y errores del modelo. Cada fila suma 100 %.

El modelo reconoce bien el viento bajo (65,1 %) y regular el medio (50,0 %). El punto débil es el viento alto: solo detecta el 18,1 % de los casos y casi siempre lo confunde con medio. Creo que pasa por dos razones. El nivel alto es el que menos datos tiene, y con solo hora, mes y estación no hay forma de distinguir una tarde de mucho viento de una tarde normal.

8. PÁGINA WEB

La página permite probar el modelo sin instalar nada. La persona escoge la hora, el mes y la estación, y la página muestra el nivel de viento más probable con el porcentaje de cada nivel. También tiene una opción para ver el cálculo paso a paso.

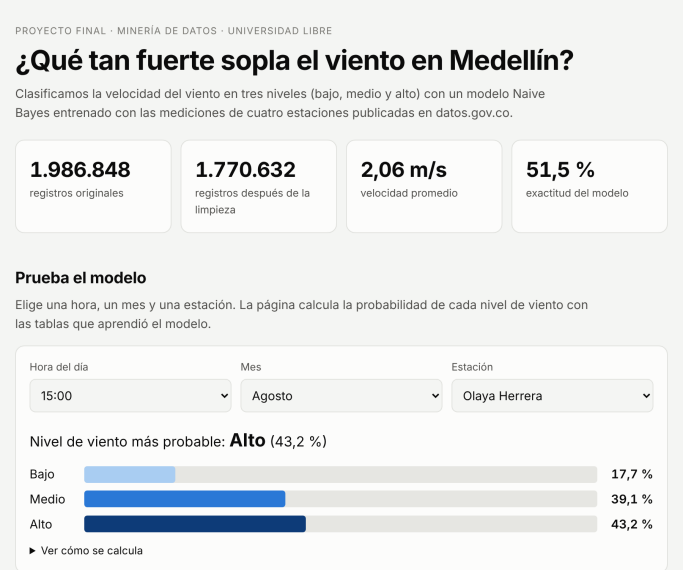


Fig. 6. Página web con el formulario para probar el modelo.

La página no necesita servidor. Como Naive Bayes es un conjunto de tablas de probabilidades, pasé esas tablas a un archivo y la página hace la multiplicación en el navegador. Comprobé que da los mismos porcentajes que el modelo en Python con los cuatro valores de prueba del código. Además de la predicción, la página muestra la tabla de limpieza, las gráficas del viento y la tabla de aciertos y errores.

9. CONCLUSIONES

La limpieza fue la parte más importante del trabajo. De casi dos millones de registros, 216.216 eran repetidos o imposibles. Si hubiera entrenado con el archivo tal como venía, Olaya Herrera habría pesado más de lo que le corresponde solo por tener dos sensores.

El modelo Naive Bayes acierta el 51,5 % frente a un 44,0 % de la respuesta más simple. No es una precisión alta, pero es lo que se puede lograr con tres variables que salen de la fecha y la estación. El modelo aprende el patrón general del viento durante el día y el año, no lo que va a pasar en una medición puntual.

La limitación más clara es el viento alto, que casi siempre se confunde con medio. El viento también depende de la temperatura, la humedad y la presión, que no están en este conjunto de datos. Un siguiente paso sería agregar esas variables medidas en las mismas estaciones para ver si el modelo distingue mejor el nivel alto.

ANEXOS

Página web para probar el modelo:
https://devharlemhm.github.io/NAIVE_BAYES_VIENTO/

Repositorio en GitHub con el código y la página:
https://github.com/devHarlemHM/NAIVE_BAYES_VIENTO

Cuaderno de trabajo en Google Colab:
https://colab.research.google.com/drive/1i134gmAOEitXlUY969o_0tnuBX78Vf-v

Fuente de los datos (datos.gov.co):
https://www.datos.gov.co/Ambiente-y-Desarrollo-Sostenible/Velocidad-del-Viento/sgfv-3yp8

CÓDIGO EN GOOGLE COLAB

```python
# PROYECTO FINAL - MINERÍA DE DATOS
# Clasificación de la velocidad del viento en Medellín con Naive Bayes
# Fuente: datos.gov.co, conjunto "Velocidad del Viento", municipio de Medellín
# https://www.datos.gov.co/Ambiente-y-Desarrollo-Sostenible/Velocidad-del-Viento/sgfv-3yp8

# 1. Importar librerías necesarias
import pandas as pd
import numpy as np
from sklearn.naive_bayes import CategoricalNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# 2. Cargar el archivo de Excel
# Tiene dos hojas porque Excel no permite más de 1.048.576 filas por hoja.
# Los códigos se leen como texto para no perder los ceros de la izquierda.
ruta = "/content/viento_medellin.xlsx"
hoja1 = pd.read_excel(ruta, sheet_name="Medellin_1", dtype={"codigoestacion": str, "codigosensor": str})
hoja2 = pd.read_excel(ruta, sheet_name="Medellin_2", dtype={"codigoestacion": str, "codigosensor": str})
datos = pd.concat([hoja1, hoja2], ignore_index=True)
print(f"Registros cargados: {len(datos)}")

# 3. Revisar qué tan sucios están los datos
print(datos.isnull().sum())
print("Filas duplicadas:", datos.duplicated().sum())
print(datos["departamento"].value_counts())
print(datos.groupby(["codigoestacion", "nombreestacion"]).size())
print(datos["valorobservado"].describe())

# 4. Limpieza de los datos
df = datos.copy()

# 4.1 Unificar los textos (había "ANTIOQUIA" y "Antioquia", "MEDELLÍN" y "Medellín")
df["departamento"] = df["departamento"].str.strip().str.upper()
df["municipio"] = df["municipio"].str.strip().str.upper()

# 4.2 Unificar el nombre de cada estación usando su código
# (una misma estación aparecía hasta con tres nombres distintos)
nombres = {
    "0027015330": "OLAYA HERRERA",
    "0027015290": "PAJARITO",
    "0027015300": "VILLANUEVA",
    "0027010810": "SANTA ELENA"
}
df["nombreestacion"] = df["codigoestacion"].map(nombres)

# 4.3 Convertir la fecha y la velocidad al tipo de dato correcto
df["fechaobservacion"] = pd.to_datetime(df["fechaobservacion"], errors="coerce")
df["valorobservado"] = pd.to_numeric(df["valorobservado"], errors="coerce")

# 4.4 Eliminar filas sin fecha, sin velocidad o sin estación
df = df.dropna(subset=["fechaobservacion", "valorobservado", "nombreestacion"])
print(f"Después de quitar nulos: {len(df)}")

# 4.5 Eliminar registros duplicados
df = df.drop_duplicates()
print(f"Después de quitar duplicados: {len(df)}")

# 4.6 Dejar una sola medición por estación y momento
# (Olaya Herrera tiene dos sensores que a veces miden al mismo tiempo)
df = df.drop_duplicates(subset=["codigoestacion", "fechaobservacion"])
print(f"Después de quitar mediciones repetidas: {len(df)}")

# 4.7 Eliminar valores imposibles
# La velocidad no puede ser negativa y más de 30 m/s (108 km/h) sería un huracán
df = df[(df["valorobservado"] >= 0) & (df["valorobservado"] <= 30)].reset_index(drop=True)
print(f"Registros limpios: {len(df)}")

# 5. Crear las variables para el modelo a partir de la fecha y la estación
codigos = {"OLAYA HERRERA": 0, "PAJARITO": 1, "VILLANUEVA": 2, "SANTA ELENA": 3}
df["hora"] = df["fechaobservacion"].dt.hour
df["mes"] = df["fechaobservacion"].dt.month
df["estacion"] = df["nombreestacion"].map(codigos)

# 6. Crear la variable objetivo con la escala de Beaufort
# 0 = Bajo (hasta 1.5 m/s), 1 = Medio (hasta 3.3 m/s), 2 = Alto (más de 3.3 m/s)
df["nivel"] = np.where(df["valorobservado"] <= 1.5, 0, np.where(df["valorobservado"] <= 3.3, 1, 2))
clases = {0: "Bajo", 1: "Medio", 2: "Alto"}
```

```
print(df["nivel"].value_counts())

# 7. Gráficas de los datos limpios
df.groupby("hora")["valorobservado"].mean().plot(marker="o", figsize=(8, 4))
plt.title("Velocidad promedio del viento por hora")
plt.xlabel("Hora del día")
plt.ylabel("Velocidad (m/s)")
plt.grid(True)
plt.show()

df.groupby("mes")["valorobservado"].mean().plot(kind="bar", figsize=(8, 4))
plt.title("Velocidad promedio del viento por mes")
plt.xlabel("Mes")
plt.ylabel("Velocidad (m/s)")
plt.show()

df["nivel"].map(clases).value_counts().plot(kind="bar", figsize=(6, 4))
plt.title("Registros por nivel de viento")
plt.ylabel("Cantidad")
plt.show()

# 8. Preparar los datos para el modelo
features = df[["hora", "mes", "estacion"]].values
labels = df["nivel"].values

# 9. Separar en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(
    features, labels, test_size=0.3, random_state=42
)

# 10. Crear y entrenar el modelo Naive Bayes
# Se usa CategoricalNB porque la hora, el mes y la estación son categorías
classifier = CategoricalNB()
classifier = classifier.fit(X_train, y_train)

# 11. Evaluar el modelo
y_pred = classifier.predict(X_test)
print(f"Precisión del modelo: {accuracy_score(y_test, y_pred):.4f}")

# Comparar con responder siempre el nivel más común (Bajo)
print(f"Precisión si siempre se responde Bajo: {(y_test == 0).mean():.4f}")

# Tabla de aciertos y errores (filas: nivel real, columnas: nivel predicho)
print(pd.crosstab(pd.Series(y_test).map(clases), pd.Series(y_pred).map(clases)))

# 12. Crear datos nuevos para probar el modelo: [hora, mes, estación]
nuevos_valores = [
    [7, 8, 0],
    [15, 8, 0],
    [14, 11, 1],
    [20, 2, 3]
]

# 13. Realizar predicciones y mostrar los resultados
predicciones = classifier.predict(nuevos_valores)
probabilidades = classifier.predict_proba(nuevos_valores)
for valor, pred, prob in zip(nuevos_valores, predicciones, probabilidades):
    print(
        f"Hora: {valor[0]}, Mes: {valor[1]}, Estación: {valor[2]} "
        f"-> Nivel de viento: {clases[pred]} ({prob.max() * 100:.1f}%)"
    )

# 14. Predecir con cualquier valor que se quiera probar
def predecir_viento(hora, mes, estacion):
    pred = classifier.predict([[hora, mes, codigos[estacion]]])[0]
    print(f"Hora: {hora}, Mes: {mes}, Estación: {estacion} -> Nivel de viento: {clases[pred]}")
    return clases[pred]

predecir_viento(16, 7, "PAJARITO")
```